Parameter-efficient transfer learning in cross-lingual scenario

Jingyan Chen

Saarland University

jich00003@uni-saarland.de

Abstract

Beyond of fine-tuning which requires an entire new model for every task, how to perform downstream tasks in a more cost-efficient way has continuously gained importance throughout the NLP field. *Prompt-tuning* and *adapters* are two closely aligned work as new lightweight and efficient fine-tuning methods. Derived from the study of employing adapter-based framework for multi-lingual transfer by Pfeiffer et al. (2020), our project mainly focus on the comparison of full and lightweight fine-tuning including adapters (Houlsby et al., 2019) and prefix-tuning (Li and Liang, 2021) in cross-lingual scenario with large/low data and in/cross-genre settings. From our evaluation and comparison, we summarize that 1) full fine-tuning obtains better performance in the cross-lingual Question Answering task with large-data setting, whereas all of them reach comparable results with smaller training samples; 2) full fine-tuning still outperforms than lightweight tuning in the cross-lingual Natural Language Inference task, but the relative performance of the former is not as good as the latter when test data from the genres unseen in the training.

1 Introduction

At present, the primary approach to carrying out downstream Natural Language Processing (NLP) tasks involves fine-tuning large pre-trained language models (LMs) (Radford et al., 2019; Devlin et al., 2019). The cross-lingual application of this paradigm presents a viable and efficient solution for the issue of low-resourced languages that lack labeled data (Conneau et al., 2018, 2020; Artetxe et al., 2020; Lewis et al., 2020). This involves training multilingual LMs on a source language and evaluating them on an unseen target language where labeled data is scarce or non-existent. However, the training of a separate LM for each specific downstream task and storing a complete set of tuned LM parameters is both computationally expensive and space-inefficient. For instance, mBART comprises 680 million parameters (Liu et al., 2020), while mT5 has anywhere from 300 million to 13 billion parameters (Xue et al., 2021).

To alleviate such a problem, *lightweight fine-tuning* has been established, freezing most of the parameters and only tuning small modules for each task. Houlsby et al. (2019) proposed “adapter", a compact and extensible model which inserts additional task-specific layers into the layers of pre-trained LMs. This approach attains comparable performance as fine-tuning the model on language understanding and generation tasks while only adding around 3% parameters (Lin et al., 2020; Houlsby et al., 2019). In addition, Li and Liang (2021) proposed “prefix-tuning" and reach strong results on Natural Language Generation tasks with learning only 0.1% of the parameters. Nevertheless, most of these parameter-efficient alternatives are conducted on monolingual data and ignore a cross-lingual setup. Lightweight tuning multilingual version of large pre-trained LMs is still not as widely studied as the monolingual ones. Inspired by this, this paper will extend lightweight tuning approaches to cross-lingual transfer learning. More precisely, this study will shed light on the following research questions:

1. In the cross-lingual question answering (QA) and Natural Language Inference (NLI) task, how does the performance change with the use of lightweight fine-tuning alternatives instead of full fine-tuning?

2. Is there a trade-off balance between the performance and other factors such as computational resources, storage space and training data size?

3. Is the relative performance of full fine-tuning and lightweight tunings different in large/low-data and in/cross-genre settings?

In order to answer these questions, we conduct experiments using the XLM-RoBERTa model (Conneau et al., 2020) for the QA and NLI tasks. Firstly, we train the models using a large-data setting (50,000 English samples) and low-data setting (50, 100, and 200 English samples) in the QA task with full fine-tuning, adapter-tuning, and prefix-tuning, and then evaluate their performance on non-English data. Next, we employ different data split methods in the NLI task to ensure the test set only contains either seen or unseen genres compared to the train set, to investigate the extrapolation performance of full fine-tuning and lightweight tuning. Finally, we compare the obtained results and analyse the potential reasons behind them, rawing conclusions based on our findings. [1] Our results indicate that: 1) full fine-tuning performs better than adapter and prefix-tuning in cross-lingual QA and NLI tasks, particularly with large-data and in-genre settings; 2) lightweight tuning could offer benefits such as parameter efficiency, computational efficiency, and modularity, making it an appropriate choice when the best performance is not a primary concern.

2 Related work

Cross-lingual transfer for QA and NLI Question Answering is the task where a model must locate the answer given a context and a question. Featured-based transfer and fine-tuning are the two most common transfer learning techniques in the NLP field (Houlsby et al., 2019). For feature-based transfer learning, Pouran Ben Veyseh (2016) used cross-lingual semantic similarity to extract the answer based on a unified semantic space among languages. More recently, the prevalent paradigm is to fine-tune pre-trained LMs, especially with the rise of multilingual evaluation sets like MLQA (Lewis et al., 2020) and XQuAD (Artetxe et al., 2020). The fine-tuned multilingual DistilBERT (Sanh et al., 2019; Abdaoui et al., 2020) obtained good results on TyDiQA (Clark et al., 2020). In addition, ByT5, a pre-trained byte-level Transformer model based on the T5 architecture (Raffel et al., 2020) proposed by Xue et al. (2022), showed outstanding performance on MLQA and TyDiQA.

Cross-Lingual Natural Language Inference is an extension to the NLI task which aims at determining the logical relationship between two text inputs. There are some data augmentation strategies to alleviate the dependence on parallel corpora like the XNLI dataset (Conneau et al., 2018). TMAN (Qi and Du, 2020) exploited adversarial training from translated data. Dong et al. (2021) managed to generate augmented data from a pre-trained sequence-tosequence model. Qi et al. (2022) proposed a novel prompt-learning based framework to reformulate the NLI task to a masked language modeling problem by constructing cloze-style questions through cross-lingual templates. Bandyopadhyay et al. (2022) introduced a novel mechanism BERT-KLD along with joint loss function based on the traditional cross-entropy loss function.

Lightweight fine-tuning During lightweight tuning, most of the pre-trained parameters are fixed and the pre-trained model is modified only with small trainable modules in downstream tasks. In prompt tuning, task description is error-prone and requires human involvement, and the effectiveness of a prompt is limited by pre-trained LMs' capacity (Lester et al., 2021). To overcome these drawbacks, Li and Liang (2021) utilized continuous prompts which were added as prefix tokens and tuned during learning on generative tasks. Lester et al. (2021) simplified this recipe and showed that prompt tuning could close the gap with fine-tuning. Another parameter-efficient approach that changes the model behaviour is "adapters", which modify the activations at any given layer instead of adding new input representations. Houlsby et al. (2019) brought this new parameter-efficient tuning approach to the NLP field with performance close to the full fine-tuned model. Artetxe et al. (2020) demployed adapters to transfer a pre-trained monolingual model to an unseen language by learning new token-level embeddings. Pfeiffer et al. (2020) used multiple adapters in a multilingual context to explicitly separate language understanding from task specification and reached competitive results in cross-lingual transfer. Built on this study, our project will explore the QA and NLI task by training XLM-RoBERTa model (Conneau et al., 2020) with different tuning methods in various settings.

3 Method

In our project, we mainly compare the adapters and prefix-tuning with full fine-tuning. This section will introduce these two lightweight tuning alternatives briefly based on the study of Houlsby et al. (2019) and Li and Liang (2021).

[1] All codes are available at https://github.com/JingyanChen22/Cross-lingual-lightweight-tuning

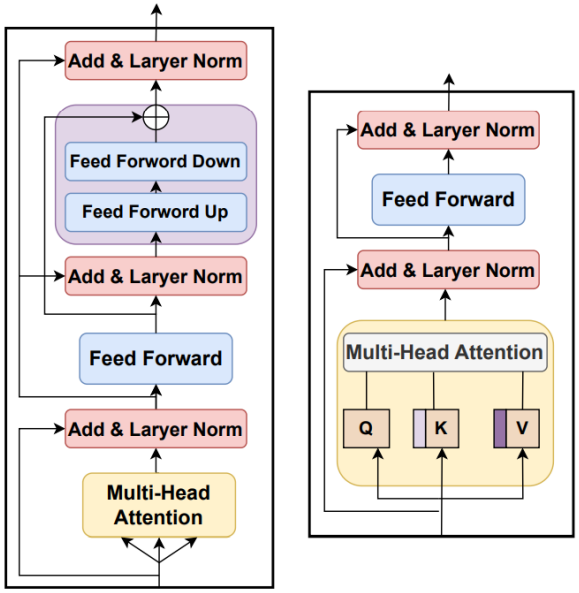


Figure 1: Left: The architecture of Bottleneck Adapter. Right: The architecture of Prefix-tuning. The purple blocks are the modified sections compared to a pre-trained Transformer-based model. The figures are referenced to AdapterHub.ml

3.1 Adapters

With the development of the study on *adapters*, the more complicated and improved adapter-based frameworks have been established such as Compacter (Karimi Mahabadi et al., 2021) and Mix-and-Match Adapters (He et al., 2021). We only utilize the basic version, Bottleneck Adapter (Houlsby et al., 2019). Generally speaking, smaller bottleneck layers (which are named "adapter") are inserted between frozen network layers in a pre-trained Transformer-based model. As illustrated in Figure 1, the bottleneck adapter network marked as purple is composed of feed-forward layers and a residual connection and is positioned after the multi-head attention layer and the feed-forward layer. To be more specific, the adapter layer projects the input h in model dimension d to a lower-dimensional space r specified by bottleneck dimension via a down-projection with $W_{down} \in \mathbb{R}^{d \times r}$, followed by a nonlinear activation function f and an up-projection with $W_{up} \in \mathbb{R}^{r \times d}$, and a residual connection surrounds these adapters (He et al., 2021).

$$\text{Adapter}(h) = h + f(hW_{down})W_{up}$$

3.2 Prefix-tuning

The intuition behind prefix-tuning is derived from prompting where prior knowledge is added through the input to affect how subsequent input is processed. Different from adapters that extend the model by additional layers, prefix-tuning prepends a sequence of continuous task-specific vectors (which are called "prefix") to the input so that multi-head attention blocks would change accordingly but leaves the activation function fixed. As the architecture illustrated in Figure 1 on the right, the prefixes are marked purple. The trainable prefix vectors P_k and $P_v \in \mathbb{R}^{l \times d}$ where d is the model dimension and l refers to the prefix length, are prepended to the original keys and values of the multi-head attention input. And then the attention mechanism is performed on the new prefixed keys and values, thus having an impact on the subsequent input (He et al., 2021). Therefore, given a sequence of vectors C we want to perform attention, and a query vector q, each head will be computed as:

$$head_i = \text{Attention}(qW_q^{(i)}, \text{concat}\left[P_k^{(i)}, CW_k^{(i)}\right], \text{concat}\left[P_v^i, CW_v^i\right])$$

4 Experiment Setup

Dataset In the QA task, the XLM-R model is trained on the Stanford Question Answering Dataset (SQuAD)[2] (Rajpurkar et al., 2016) and evaluated on the Cross-lingual QA Dataset (XQuAD)[3] (Artetxe et al., 2020). The SQuAD dataset consists of more than 100,000 English question-answer pairs posed by crowd workers based on Wikipedia articles and is split into the train and evaluation sets. The XQuAD is the cross-lingual version converted from the SQuAD dataset to 11 non-English languages and it only contains 1,190 question-answer pairs for each language. Due to limited computational resources, only 50,000 out of 87,599 samples and 6,250 out of 10,570 samples from the original SQuAD dataset are randomly selected as the train and validation sets, respectively [4]. For each sample in the train and validation sets, most questions have one or three answers and the average length of each answer is about 20 English words. It is difficult to obtain generalised statistics for the test set as it contains data in different target languages, but there is only one answer per sample for all languages so that

[2] https://rajpurkar.github.io/SQuAD-explorer/

[3] https://github.com/deepmind/xquad

[4] The preliminary experiment has shown that the model indeed could learn this specific task with a such relatively smaller amount of training data.

our trained models will only generate one single answer during the evaluation.

For the NLI task, we use Multi-Genre Natural Language Inference (MultiNLI) (Williams et al., 2018)[5] corpus for training, which is a crowd-sourced collection of 433k sentence pairs annotated with textual entailment information covered five different genres. The test set from the Cross-lingual Natural Language Inference (XNLI) (Conneau et al., 2018)[6] corpus is used for evaluating. It is an extension of the MultiNLI and contains about premise-hypothesis pair translations in 14 languages (5,000 pairs for each language). Besides the five genres that are already covered in the train set from the MultiNLI, it still includes another additional unseen five genres and we will construct an extrapolation setting to experiment with based on this. For both the MultiNLI and the XNLI corpus, the numbers of entailment, contradiction and neutral labels are close and in balance. The model is trained on all samples from the train set without selecting unlike in the QA task.

Note that we select only five non-English target languages (Arabic, Chinese, German, Thai and Turkish) coming from different language families to obtain as representative cross-lingual performance as possible for both of the QA and NLI task, ans we will analyse their performance difference with regard to some linguistic factors in Section 5.1.1.

Metrics We use the official evaluation script of the SQuAD dataset: Exact Match (EM) and F1 score, which are dominant metrics for the QA task. As the name suggests, EM is a strict all-or-nothing metric that assigns a score of 1 if the characters in the model's prediction exactly match those in the gold answer; otherwise, the score is 0. The number of shared words between the prediction and the gold answer is the basis of the F1 score. Consequently, EM scores are usually slightly less reliable and much lower than F1 scores as Table 1 shows, and the result report in Section 5.1 mainly analyses F1 scores.

As for the NLI task, we use the typical metrics of accuracy, F1-score and Matthews Correlation Coefficient (MCC). In the context of NLI, MMC can evaluate the performance of a model that predicts the relationship between two pieces of text and provide a more nuanced and robust evaluation of a model's performance compared to accuracy and F1-score. This is because MCC considers all four categories (true positives, true negatives, false positives, and false negatives), making it less affected by class imbalance and the different cost of false positives and false negatives.

Model The XLM-RoBERTa base model (Conneau et al., 2020) is utilized for the QA and NLI task. It is a multilingual version of RoBERTa and is pre-trained on 2.5TB of filtered CommonCrawl data across 100 languages. The target languages in our case are all included in its pre-training.

Training procedure The detailed hyperparameter settings are listed in Appendix A. All models are trained with the same hyper-parameter settings per task except for the number of epochs. For adapters and prefix-tuning, we train another model for double number of epochs since they involve far fewer updating parameters than full fine-tuning and a few more training epochs may be necessary to make a more complete comparison. Note that the purpose of this project is to compare full fine-tuning and lightweight tuning rather than improve the model's performance; thus, the step of hyperparameter optimization will be skipped.

5 Results & Analysis

The following section presents the results of our cross-lingual evaluation, which utilizes full fine-tuning, adapter-tuning, and prefix-tuning for the QA and NLI tasks. The objective is to compare the cross-lingual performance of these three training methods under different settings.

5.1 Question Answering

We have created two data settings with vastly different quantities. The performance of models trained with the large-data setting (50,000 training samples) is displayed in Table 1, while the average scores for each target language, based on Table 1 and Figure 2, are demonstrated in Table 2. The results will be presented from two perspectives: 1) a comparison of performance between the three tuning methods; 2) a comparison of performance between the five non-English target languages.

5.1.1 Large-data setting

At three epochs, the full fine-tuned model performs best among these three tuning approaches, getting the highest average EM score of 51.90% and F1 score of 65.17%. The results of adapter-tuning and

[5] https://cims.nyu.edu/~sbowman/multinli/
[6] https://cims.nyu.edu/~sbowman/xnli/

	Epoch	EN		AR		DE		TH		TR		ZH		Avg	
		EM	F1	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1
Full fine-tune	3	**69.92**	**80.85**	**39.24**	**57.98**	**55.97**	**70.96**	**53.53**	**63.98**	**49.24**	**65.99**	**43.53**	**51.28**	**51.90**	**65.17**
Adapter-tune	3	64.71	75.85	37.98	54.87	49.50	65.90	51.26	60.70	43.87	59.58	38.32	47.26	47.61	60.69
	6	67.31	78.76	38.07	55.25	51.09	68.02	**53.53**	63.29	48.40	63.61	40.25	49.85	49.78	63.13
Prefix-tune	3	65.38	76.38	36.81	55.03	50.08	65.97	52.86	61.99	43.61	59.71	37.39	46.58	47.69	60.94
	6	67.31	78.65	37.23	55.77	54.45	68.74	52.77	61.89	44.54	61.28	38.40	46.16	49.12	62.08

Table 1: EM and F1 scores (%) of the XLM-RoBERTa model trained on 50,000 samples using three tuning methods (Fine-tuning, Adapter-tuning and Prefix-tuning) evaluated on test data in six different target languages: English (EN), Arabic (AR), German (DE), Thai (Th), Turkish (TR) and Chinese (ZH). The best scores for each target language are boldfaced.

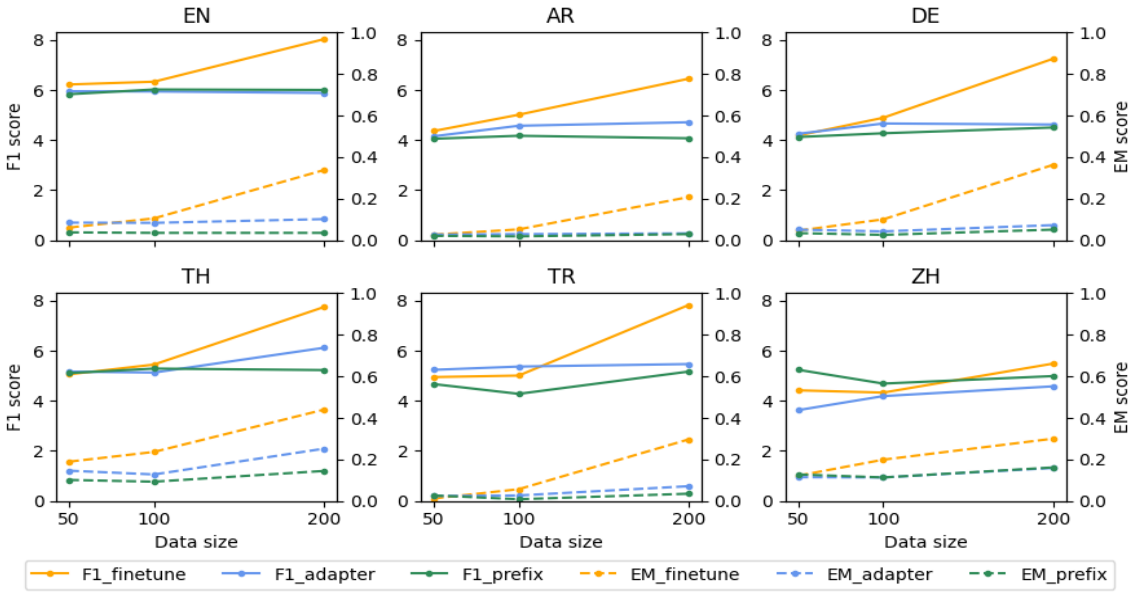


Figure 2: EM and F1 scores (%) with the low-data setting of size {50, 100, 200} evaluated on test data in English (EN), Arabic (AR), German (DE), Thai (Th), Turkish (TR) and Chinese (ZH).

	EN		AR		DE		TH		TR		ZH	
	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1
Large-data	66.93	78.10	37.87	55.78	52.22	67.32	52.79	62.37	45.93	62.03	39.58	48.23
Low-data	0.81	6.24	0.41	4.62	0.72	4.75	1.59	5.59	0.51	5.33	1.3	4.62

Table 2: Average EM and F1 scores (%) in large and low-data settings evaluated on test data in six different target languages (built on Table 1 and Figure 2).

prefix-tuning are almost the same, and their average EM and F1 scores are close to 48% and 61%, respectively. When adapter and prefix-tuning are trained for six epochs, their average performance increases by about 2% to 3%. but still lag behind the full fine-tuned model despite they are trained for a longer time. In terms of lightweight tuning, adapters perform slightly better than prefix-tuning by less than 1% on average. Overall, with adding much fewer task-specific parameters, adapter and prefix-tuning are effective to understand a context and answer questions based on this context even cross-lingually. However, with the same training time, we can see that they still exhibit an obvious cross-lingual performance gap (four to five points) compared to full fine-tuning. Longer train-

ing time indeed leads to better performance for lightweight tuning, but it raises the question of whether it is worthwhile to spend almost twice the time to achieve only a marginal improvement. We will further elaborate on this trade-off in Section 6.

As expected, the average EM and F1 scores evaluated on English data achieve the highest with 66.93% and 78.10%, respectively. Among other target languages, the general performance ranking from the best to worst is as follows: German, Thai, Turkish, Arabic and Chinese. The performance difference between non-English and English ranges from 10% (German) to 30% (Chinese). De Vries et al. (2022) have proved that the factors including matching language families, writing systems, word order systems, and lexical-phonetic distance significantly impact cross-lingual performance. According to the aforementioned ranking, we will analyze the results for each language individually considering these linguistic factors.

The first-ranked language is German, which is not surprising since it belongs to the Germanic branch of the Indo-European language family and shares the same alphabet and many similar words with English. Turkish, on the other hand, is from the Turkic family and has a subject-object-verb sentence structure that differs from English. However, the Turkish texts in the XQuAD dataset are written in Latin script like English, which may have contributed to its relatively good performance. Arabic and Chinese, which have entirely different writing systems and scripts from English, performed much worse than the others, with Chinese having the lowest score. Arabic uses its unique script, an abjad system, while Chinese uses a logographic system. Additionally, Chinese has no explicit whitespace tokenization, which may have negatively impacted its evaluation results. The high scores of Thai are unexpected since it is a distant language with its own distinct linguistic features compared to English. We can only hypothesize that Thai itself is relatively easier than other target languages.

Nonetheless, in terms of the close scores in the second row of Table 2, we can see that the low-data setting would probably limit the role of these linguistic factors. Therefore, we are supposed to consider the similarity of the source and target languages when performing cross-lingual downstream tasks with a large amount of data, no matter which tuning method is deployed.

5.1.2 Low-data setting

Li and Liang (2021) argue that prefix-tuning outperforms fine-tuning in low-data regimes for the table-to-text task and is able to generate more faithful texts. To reconfirm this claim, we replicate their experimental approach by subsampling the full dataset to create smaller datasets of size {50, 100, 200}. For each size, we sample the data with three random seeds and train the models with two random training seeds, averaging over six models to get a reliable estimate. We train the models for three epochs like in large-data setting since preliminary experiments indicated no performance gain with longer training time in the low-data setting.

From Figure 2, we can observe that the performance of adapter-tuning and prefix-tuning is almost identical and fine-tuning still outperforms them in most cases, as seen in the large-data setting. Combining EM and F1 scores, the performance of all three tuning methods is comparable when the amount of data is only 50 and 100. However, the advantage of full fine-tuning becomes more apparent as the dataset size increases to 200, except on the Chinese test set. This suggests that fine-tuning is more sensitive to changes in sample size. More data could boost its performance directly even if the data size is not large enough, whereas adapter and prefix-tuning require sufficient data to achieve satisfying performance. These results are different from the findings of Li and Liang (2021), raising further questions about the effectiveness of lightweight tuning in cross-lingual scenarios. It is also worth noting that the performance differences between English and non-English languages in the low-data setting are not distinguishable, with a maximum difference of only two points.

5.2 Natural Language Inference

We experiment with in-genre and cross-genre settings in the NLI task to further compare the extrapolation performance of each training methods. In the in-genre setting, the test set would only include the genres that have already seen during the training ({*government, telephone, fiction, travel, slate*}). While in the cross-genre setting, the test set would not closely resemble any seen at training time and contain totally non-overlapping genres ({*facetoface, letters, nineeleven, oup, verbatim*}). We will report the general comparison results in both of the settings first, and then analyse the performance difference in each setting.

	Epoch	In-genre			Cross-genre			Difference		
		Acc	F1	MCC	Acc	F1	MCC	Acc	F1	MCC
Full fine-tune	1	**74.97**	**74.90**	**62.94**	**74.77**	**74.69**	**62.68**	*0.20*	*0.21*	*0.26*
Adapter-tune	1	68.98	68.84	53.88	68.95	68.83	53.79	0.03	0.01	0.09
	2	70.78	70.62	56.66	70.03	69.88	55.52	(0.75	0.74	1.14)
Prefix-tune	1	70.52	70.33	56.44	70.35	70.19	56.21	0.17	0.14	0.23
	2	70.95	70.66	57.19	70.70	70.44	56.84	(0.25	0.22	0.35)

Table 3: The average accuracy, F1-score and MCC over all target languages in the in/cross-genre settings using full fine-tuning, adapter-tuning and prefix-tuning. The best results are baldfaced. *Difference* represents the relative performance gap of each tuning method between the in-genre and cross-genre settings. The biggest differences are shown in italics. Note that the values in brackets are not included in this comparison due to longer training time.

Table 3 summarizes our main results with two settings. We do not show the detailed results for each target languages like in the QA task but average their scores to make the table more concise. As expected, the results in the in-genre setting are better than the cross-genre one overall. With training for one epoch, full fine-tuning still outperforms adapters and prefix-tuning at least four points in all metrics in both of the settings. The longer double training time brings quite slight increase for lightweight tuning, but their performance is still worse than full fine-tuning. Unlike the previous task, the results of prefix-tuning are better than adapters, especially with training for one epoch. Once again, unfortunately, contrary to the findings of Li and Liang (2021), neither adapter-tuning nor prefix-tuning has any significant performance gain in such extrapolation settings. Accounting for this, we attend to their relative instead of absolute performance comparison next.

We also display the relative performance difference between the in-genre and cross-genre settings in Table 3 to further analyse the generalization issue during model training. Full fine-tuning involves updating all pre-trained parameters for downstream tasks, whereas prefix-tuning and adapter-tuning preserve them. Since LMs are pre-trained on general-purpose corpus, preserving the LM parameters may aid generalization to unseen domains. While lightweight tuning performs worse than full fine-tuning in terms of absolute scores in the cross-genre setting, the relative performance gap between the in-genre and cross-genre settings is smaller for lightweight tuning than for full fine-tuning when training epochs are similar. This suggests that lightweight tuning may have a better generalization capability. Increasing the training time for adapters and prefix-tuning results in larger performance gaps, indicating that longer training can lead to overfitting and reduced generalization. Overall, we assume that parameter-efficiency could keep the pre-trained LMs intact as much as possible, thus improving the ability to generalize. Updating the whole pre-trained parameters is likely to make the LMs suffer from catastrophic forgetting and poor generalization if training time is not carefully managed.

6 Discussion

Based on the reported results, this section will discuss the trade-off between effectiveness and efficiency. The pros and cons of full and lightweight fine-tuning are analysed from the following aspects:

- **Training procedure:** Parameter efficiency is an outstanding strength of lightweight tuning. Its average training time per epoch in our case is about 0.86h in the QA task while the full fine-tuning takes around one hour. In the NLI task, the difference in time taken is much greater and full fine-tuning consumes nearly twice as much time per epoch.

- **Application procedure:** Lightweight tuning is more modular because the updater parameter subset can be extracted and shared independently of the base model. This also saves much storage space in comparison with saving the whole fine-tuned model. In our case, for instance, the fine-tuned XLM-RoBERTa occupies more than 1GB of storage space, whereas the trained adapters only needs about 40MB. On top of this, its composability could bring us unexpected good results. For example, we

can stack or mix the task-specific adapters trained from different but close tasks like Dependency Parsing and Part-of-speech Tagging, to leverage the combined knowledge.

- **Performance:** Lightweight tuning provides on-par performance in NLG tasks by adding only about 3% or even 0.1% training parameters (Li and Liang, 2021), but not in our cross-lingual case. It still underperforms fine-tuning with double training time in either case. However, the performance of lightweight tuning on out-of-domain test data is relatively better. This might be because only a small percentage of parameters is updated during training, but this is still an open question and deserves more studies in the future.

In summary, the preference for full and lightweight fine-tuning varies when considering different aspects. Full fine-tuning would be a right selection if the goal is to attain better performance and sufficient data is available; otherwise, lightweight tuning should be chosen due to its cost-efficiency and potential good generalization capability. As pre-trained LMs have expanded from millions to billions or trillions of parameters, approaches like adapters, whose representative characteristics are efficiency and convenience, have become even more essential. However, its effectiveness have yet to be examined in more NLP tasks and scenarios like the cross-lingual setting.

7 Conclusion & Future work

This project evolves around the comparison of full and lightweight fine-tuning in a multi-lingual scenario, to figure out a computationally efficient but excellently performing training method. We discover that full fine-tuning performs better than adapter and prefix-tuning in the cross-lingual QA and NLI task with the specific corpora, but both of them have their internal (dis)advantages that are worth weighing when making a choice.

How to boost the development of low-resourced languages in the NLP field via leveraging the cross-lingual pre-trained LMs is one of the most critical and challenging tasks. Hence, future work should also concentrate on making full use of parameter-efficient tuning and enhancing its effects in zero-shot or few-shot cross-lingual transfer, to mitigate the data availability issue for low-resourced languages.

Limitations

From the results with both of the large-data and low-data settings in Table 2, we can see that the performance still has much room for improvement. The reasons might be: 1) GPU resources are limited so that not all data has been utilized; 2) hyperparameter optimization has been skipped. Additionally, comparative results derived from only the QA and NLI task might be not sufficient to make reliable conclusions. More tasks and models could be attempted if conditions permit.

References

Amine Abdaoui, Camille Pradel, and Grégoire Sigel. 2020. Load what you need: Smaller versions of multilingual bert. *arXiv preprint arXiv:2010.05609*.

Mikel Artetxe, Sebastian Ruder, and Dani Yogatama. 2020. On the cross-lingual transferability of monolingual representations. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics.

Dibyanayan Bandyopadhyay, Arkadipta De, Baban Gain, Tanik Saikh, and Asif Ekbal. 2022. A deep transfer learning method for cross-lingual natural language inference. In *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, pages 3084–3092, Marseille, France. European Language Resources Association.

Jonathan H. Clark, Eunsol Choi, Michael Collins, Dan Garrette, Tom Kwiatkowski, Vitaly Nikolaev, and Jennimaria Palomaki. 2020. TyDi QA: A benchmark for information-seeking question answering in typologically diverse languages. *Transactions of the Association for Computational Linguistics*, 8:454–470.

Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2020. Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8440–8451, Online. Association for Computational Linguistics.

Alexis Conneau, Ruty Rinott, Guillaume Lample, Adina Williams, Samuel Bowman, Holger Schwenk, and Veselin Stoyanov. 2018. XNLI: Evaluating cross-lingual sentence representations. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2475–2485, Brussels, Belgium. Association for Computational Linguistics.

Wietse De Vries, Martijn Wieling, and Malvina Nissim. 2022. Make the best of cross-lingual transfer: Evidence from POS tagging with over 100 languages.

In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7676–7685, Dublin, Ireland. Association for Computational Linguistics.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

Xin Dong, Yaxin Zhu, Zuohui Fu, Dongkuan Xu, and Gerard de Melo. 2021. Data augmentation with adversarial training for cross-lingual NLI. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 5158–5167, Online. Association for Computational Linguistics.

Junxian He, Chunting Zhou, Xuezhe Ma, Taylor Berg-Kirkpatrick, and Graham Neubig. 2021. Towards a unified view of parameter-efficient transfer learning. *arXiv preprint arXiv:2110.04366*.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2790–2799. PMLR.

Rabeeh Karimi Mahabadi, James Henderson, and Sebastian Ruder. 2021. Compacter: Efficient low-rank hypercomplex adapter layers. *Advances in Neural Information Processing Systems*, 34:1022–1035.

Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3045–3059, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Patrick Lewis, Barlas Oguz, Ruty Rinott, Sebastian Riedel, and Holger Schwenk. 2020. MLQA: Evaluating cross-lingual extractive question answering. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7315–7330, Online. Association for Computational Linguistics.

Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning: Optimizing continuous prompts for generation. *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, abs/2101.00190.

Zhaojiang Lin, Andrea Madotto, and Pascale Fung. 2020. Exploring versatile generative language model via parameter-efficient transfer learning. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 441–459, Online. Association for Computational Linguistics.

Yinhan Liu, Jiatao Gu, Naman Goyal, Xian Li, Sergey Edunov, Marjan Ghazvininejad, Mike Lewis, and Luke Zettlemoyer. 2020. Multilingual denoising pre-training for neural machine translation. *Transactions of the Association for Computational Linguistics*, 8:726–742.

Jonas Pfeiffer, Ivan Vulić, Iryna Gurevych, and Sebastian Ruder. 2020. MAD-X: An Adapter-Based Framework for Multi-Task Cross-Lingual Transfer. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7654–7673, Online. Association for Computational Linguistics.

Amir Pouran Ben Veyseh. 2016. Cross-lingual question answering using common semantic space. In *Proceedings of TextGraphs-10: the Workshop on Graph-based Methods for Natural Language Processing*, pages 15–19, San Diego, CA, USA. Association for Computational Linguistics.

Kunxun Qi and Jianfeng Du. 2020. Translation-based matching adversarial network for cross-lingual natural language inference. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(05):8632–8639.

Kunxun Qi, Hai Wan, Jianfeng Du, and Haolan Chen. 2022. Enhancing cross-lingual natural language inference by prompt-learning from cross-lingual templates. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1910–1923, Dublin, Ireland. Association for Computational Linguistics.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, Austin, Texas. Association for Computational Linguistics.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*.

Adina Williams, Nikita Nangia, and Samuel Bowman. 2018. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 1112–1122, New Orleans, Louisiana. Association for Computational Linguistics.

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. 2022. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306.

Linting Xue, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. 2021. mT5: A massively multilingual pre-trained text-to-text transformer. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 483–498, Online. Association for Computational Linguistics.

A Hyperparameters

In Table 4, we report the hyperparameters used to train the models documented in the Experiment Setup section.

Parameter	**QA**	**NLI**
batch size	16	16
learning rate	7e-5	3e-5
lr scheduler type	linear	linear
warm-up step	100	-
warm-up ratio	-	0.06
epoch	3 or 6	1 or 2
optimizer	AdamW	AdamW
evaluation strategy	steps	steps

Table 4: Hyperparameter settings for the XLM-RoBERTa model in the QA and NLI task.